

Primer Modelo

Nombre: Steven Chuiza

Fecha: 14/05/2026

```
In [8]: import numpy as np
from sklearn import datasets
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
In [9]: # Carga de datos
iris = datasets.load_iris()
print(iris.target_names)
```

['setosa' 'versicolor' 'virginica']

```
In [10]: # Mostrar características de la tabla de datos.
print("Tabla de datos: %d instancias y %d atributos" % (iris.data.shape[0], iris
print("Valores de la clase:", set(iris.target))

# Cuantificamos el número de instancias que contiene el dataset por clase
valores, ocurrencias = np.unique(iris.target, return_counts=True)
print(valores, ocurrencias)
```

Tabla de datos: 150 instancias y 4 atributos

Valores de la clase: {np.int64(0), np.int64(1), np.int64(2)}

[0 1 2] [50 50 50]

```
In [11]: # Test: hold-out split 80-20%. # Partición externa
X_training, X_test, y_training, y_test = train_test_split(iris.data, iris.target
valores_test, ocur_test = np.unique(y_test, return_counts=True)
print('Test: ', 'clases:', valores_test, ' ocurrencias: ', ocur_test)
```

Test: clases: [0 1 2] ocurrencias: [10 9 11]

```
In [12]: # Estandarizar las características de entrenamiento y de test
standardizer = StandardScaler()
X_training = standardizer.fit_transform(X_training)
X_test = standardizer.transform(X_test)
```

```
In [13]: # Validación: hold-out split 80-20%. # Partición interna
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_s
valores_train, ocur_train = np.unique(y_train, return_counts=True)
print('Entrenamiento: ', 'clases:', valores_train, ' ocurrencias:', ocur_train

valores_val, ocur_val = np.unique(y_val, return_counts=True)
print('Validation: ', 'clases:', valores_val, ' ocurrencias:', ocur_val)
```

Entrenamiento: clases: [0 1 2] ocurrencias: [32 30 34]

Validation: clases: [0 1 2] ocurrencias: [8 11 5]

```
In [14]: # Construcción del objeto que contiene el algoritmo de aprendizaje.
clf = DummyClassifier(strategy='prior', random_state=42)
```

```
In [15]: # Entrenamiento del algoritmo de aprendizaje.
clf = clf.fit(X_train, y_train)
```

```
In [16]: # Evaluación del algoritmo de aprendizaje con el método "score" que devuelve dir
val_accuracy = clf.score(X_val, y_val)
print("Exactitud en validación: ", val_accuracy)

test_accuracy = clf.score(X_test, y_test)
print("Exactitud en test: ", test_accuracy)
```

Exactitud en validación: 0.20833333333333334

Exactitud en test: 0.36666666666666664

```
In [17]: # Obtenemos las predicciones sobre conjunto de validación y de test
y_pred_val = clf.predict(X_val)
print('Predicciones de validación ', y_pred_val)
print('Etiquetas reales validación', y_val)

y_pred_test = clf.predict(X_test)
print('\nPredicciones de test ', y_pred_test)
print('Etiquetas reales test', y_test)
```

Predicciones de validación [2 2]

Etiquetas reales validación [1 1 0 0 0 2 1 2 2 2 1 1 1 1 1 0 2 0 1 0 1 1 0 0]

Predicciones de test [2 2]

Etiquetas reales test [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]

```
In [18]: # Aplicamos un ejemplo con un clasificador más complejo que el "dummyclassifier"
from sklearn.svm import SVC
svc = SVC(C=0.5) # Definimos algoritmo
svc.fit(X_train, y_train) # Entrenamos modelo

val_accuracy = svc.score(X_val, y_val) # Evaluamos modelo en validación
print('Exactitud en validación ', np.round(val_accuracy*100, 4), '%')

test_accuracy = svc.score(X_test, y_test) # Evaluamos modelo en test
print('Exactitud en test ', np.round(test_accuracy*100, 4), '%')
```

Exactitud en validación 91.6667 %

Exactitud en test 96.6667 %

```
In [19]: # Obtenemos las predicciones sobre conjunto de validación y de test
y_pred_val = svc.predict(X_val)
print('Predicciones de validación ', y_pred_val)
print('Etiquetas reales validación', y_val)

y_pred_test = svc.predict(X_test)
print('\nPredicciones de test ', y_pred_test)
print('Etiquetas reales test', y_test)
```

Predicciones de validación [1 1 0 0 0 2 2 2 2 2 1 2 1 1 1 0 2 0 1 0 1 1 0 0]

Etiquetas reales validación [1 1 0 0 0 2 1 2 2 2 1 1 1 1 1 0 2 0 1 0 1 1 0 0]

Predicciones de test [1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]

Etiquetas reales test [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]

```
In [22]: # Guardar modelo
import pickle
with open('models/model.pkl', 'wb') as fw:
    pickle.dump(svc, fw)

# Cargar modelo
with open('models/model.pkl', 'rb') as fr:
    modelo_cargado = pickle.load(fr)
print ("Modelo cargado")

# Guardamos el escalado
with open('models/scaler.pkl', 'wb') as fw:
    pickle.dump(standardizer, fw)

# cargamos el scaler
with open("models/scaler.pkl", "rb") as f:
    scaler_cargado = pickle.load(f)

print("Modelo y scaler cargados")
```

Modelo cargado
Modelo y scaler cargados

```
In [23]: #Nuevos datos
nuevo_dato = [[7.5, 0.5, 3.4, 1.2]]
#Escala lo datos
nuevo_dato_escalado = scaler_cargado.transform(nuevo_dato)
print(nuevo_dato_escalado)
```

```
[[ 2.06107356 -5.72762329 -0.18719854  0.02224751]]
```

```
In [25]: # Predicción
prediccion = svc.predict(nuevo_dato_escalado)
print(prediccion)
```

```
[2]
```

Actividad

```
In [34]: # 1. Ingresar los nuevos parámetros por teclado
print("Ingrese los datos de la flor:")
sepal_length = float(input("Ingrese el tamaño del sepalo o Sepal Length: "))
sepal_width = float(input("Ingrese la anchura del sepalos o Sepal Width: "))
petal_length = float(input("Ingrese el tamaño del petalo o Petal Length: "))
petal_width = float(input("Ingrese la anchura del petalo o Petal Width: "))
```

Ingrese los datos de la flor:

```
In [35]: # Creamos la lista con los datos ingresados
nuevo_dato = [[sepal_length, sepal_width, petal_length, petal_width]]

# 2. Escalar los datos ingresados
nuevo_dato_escalado = scaler_cargado.transform(nuevo_dato)
print(nuevo_dato_escalado)
```

```
[[ -6.83740077  0.98006827 -2.02098015 -1.31260282]]
```

```
In [36]: # 3. Realizar la predicción
prediccion = modelo_cargado.predict(nuevo_dato_escalado)
print(f"\nResultado de la predicción: {prediccion}")
```

Resultado de la predicción: [2]

```
In [38]: # 4. Condicional para mostrar el nombre de la especie
if (prediccion == 0):
    print("La flor en base a los datos ingresados es Setosa")
elif (prediccion == 1):
    print("La flor en base a los datos ingresados es Versicolor")
else:
    print("La flor en base a los datos ingresados es Virginica")
```

La flor en base a los datos ingresados es Virginica

Repositorio de github: https://github.com/tiven29/Fundamentos_Machine_Learning

In []: